

SLE-3: Architectural Design (Full C4 Model)

Course: 02AML204 — Introduction to Artificial Intelligence

PRN: 25UAM111

Name: Parshwa Ramesh Bhavare

Division: B

Date: 04 October 2026

GitHub / Portfolio Link (optional): <https://github.com/ParshwaBhavare?tab=repositories>

1. System Title & Short Description

BFS vs DFS Graph Traversal Profiling System (continued from SLE-2). The system builds fixed test graphs (a 10-vertex tree, K10 and K20), runs Breadth-First Search and Depth-First Search on the same adjacency-list graph from vertex 0, times them using 3 runs of 10,000 traversals each, and presents the results as tables and charts. It solves the problem of comparing two search strategies fairly: the same input, the same start vertex and the same timing method for both algorithms.

2. Context Diagram (Level 1)

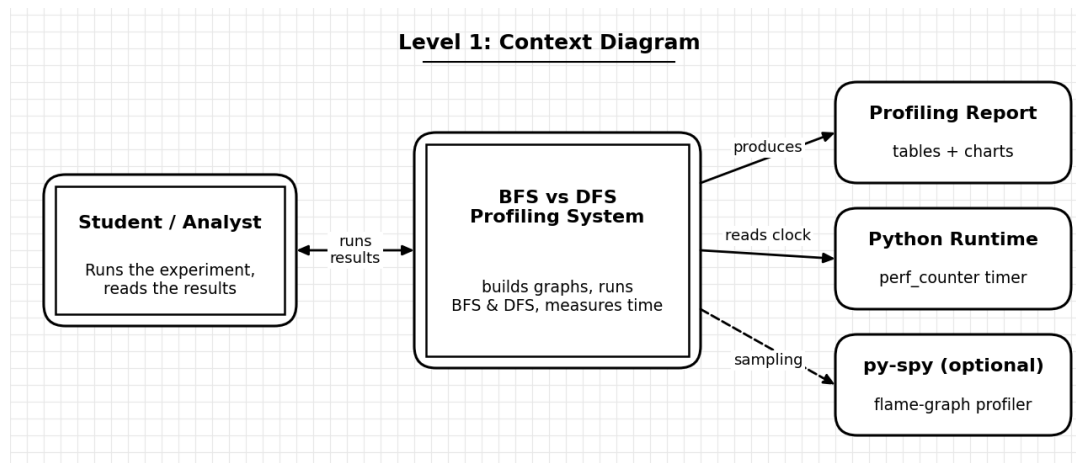


Fig. 1 — Level 1: System Context

The Student / Analyst is the only user: they choose the graph and the algorithm and read the results. The system depends on the Python runtime for its timer (`time.perf_counter`). `py-spy` is an optional external tool used to capture flame graphs. The final output is the profiling report with the comparison tables and charts.

3. Container Diagram (Level 2)

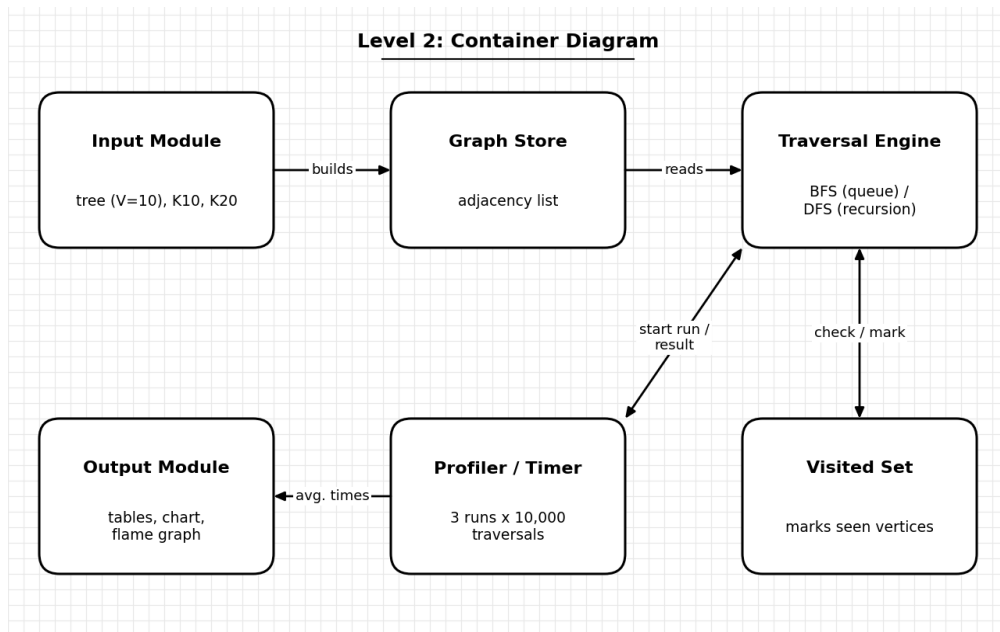


Fig. 2 — Level 2: Containers inside the system

- **Input Module**: creates the deterministic test graphs (tree V=10, K10, K20) for the Best / Average / Worst cases.
- **Graph Store**: keeps the graph as an adjacency list so both algorithms read exactly the same data.
- **Traversal Engine**: runs BFS (FIFO queue) and DFS (recursion) from start vertex 0. This is the core container.
- **Visited Set**: remembers visited vertices so each vertex is processed once and loops are avoided.
- **Profiler / Timer**: repeats each traversal 10,000 times, runs it 3 times, and averages the batch times.
- **Output Module**: produces the summary table, the log-scale time chart and the flame-graph view.

4. Component Diagram (Level 3) — Traversal Engine

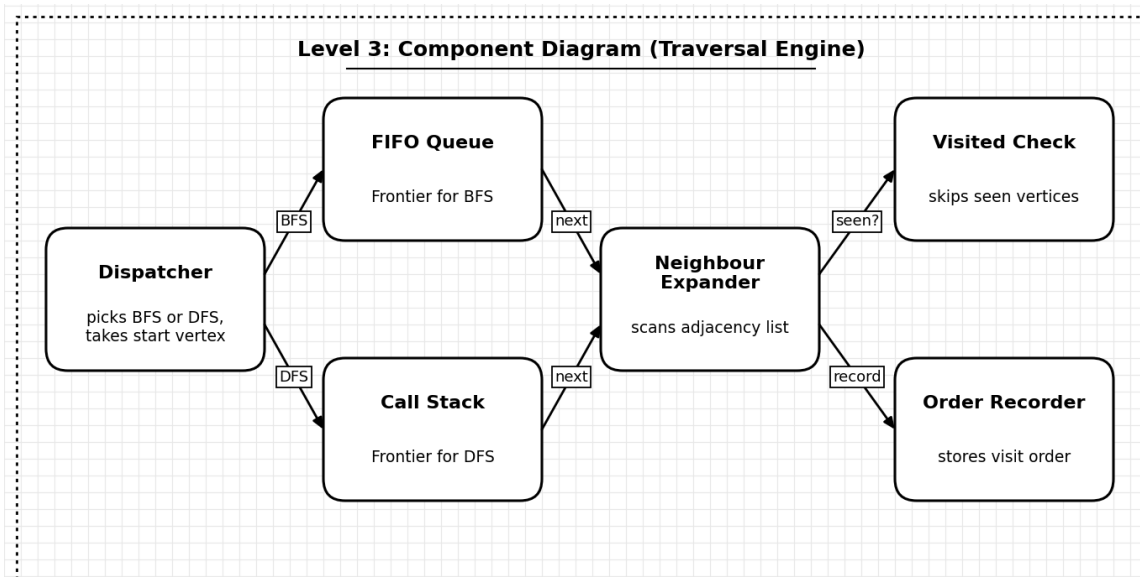


Fig. 3 — Level 3: Components inside the Traversal Engine

The Dispatcher receives the start vertex and decides whether BFS or DFS runs. BFS keeps its frontier in a FIFO Queue and DFS keeps it on the Call Stack. Both give the next vertex to the Neighbour Expander, which scans that vertex's adjacency list and uses the Visited Check to skip vertices already seen. The Order Recorder stores the sequence of vertices so the two traversal orders can be compared.

5. Code Level Overview (Level 4)

Main classes / functions (names and one-line job only):

- **class Graph** adjacency-list graph with V vertices and `add_edge(u, v)`.
- **def build_tree()** / **build_complete(n)** create the 10-vertex tree, K10 and K20 test graphs.
- **def bfs(graph, start)** queue-based traversal; returns the visit order.
- **def dfs(graph, start)** recursive traversal using a visited array; returns the visit order.
- **def time_traversal(fn, graph)** runs 10,000 traversals in one batch using `time.perf_counter`.
- **def profile_all()** runs 3 batches per case and algorithm, then averages them.
- **def make_report()** builds the tables, the log-scale chart and the flame-graph view.

6. Design Decisions

The system is split into small modules so that graph creation, traversal, timing and reporting can change independently. BFS and DFS share one Graph Store and one Visited Set design, which keeps the comparison fair. The Profiler is kept outside the Traversal Engine so that timing code does not mix with algorithm code. The Traversal Engine was chosen for the Level 3 diagram because it holds the real search logic. No heuristic module is shown because neither BFS nor DFS uses one.

7. AI Contribution Note

- **AI tools used:** Claude (Anthropic).
- **What AI helped with:** suggesting the C4 structure, drawing the three diagrams, and writing the first draft of the explanations in the required format.
- **What I did myself:** chose the BFS vs DFS system from my SLE-2, checked that every container and function matches my own code and profiling work, and reviewed and edited the final text.

8. Conclusion

C4 helped me look at the same BFS vs DFS project from four zoom levels: who uses it, what the main parts are, what is inside the Traversal Engine, and which functions implement it. I learned that a clear boundary between traversal logic, data, timing and reporting makes a project easier to explain, test and extend, for example by adding A* or a new graph type later without touching the other parts.